

目录

- TOC {toc}

本章介绍了深度学习的应用案例研究，深度学习是使用人工神经网络的机器学习的一个新近分支。机器学习已在许多应用领域中使用，根据从数据集中提取的经验来训练或调整应用逻辑。为了有效，通常需要用大量的数据进行此类训练。虽然机器学习作为计算机科学的主题已经存在很长时间了，但由于两个原因，它最近在实际工业中获得了极大的接受度。第一个原因是因特网的广泛使用带来的大量数据。第二个原因是价格便宜、大规模并行的GPU计算系统，能够有效地用这些大型数据集训练应用逻辑。我们将从机器学习和深度学习的简要介绍开始，然后更详细地考虑最受欢迎的深度学习算法之一：卷积神经网络（CNN）。CNN具有高计算对内存访问比率和高并行度，这使它们成为GPU加速的理想候选。我们首先介绍卷积神经网络的基本实现。接下来，我们将展示如何通过共享内存改进这个基本实现。然后，我们将展示如何将卷积层表述为矩阵乘法，这可以通过使用现代GPU中高度优化的硬件和软件来加速。

16.1 背景

机器学习，这个术语是由IBM的阿瑟·塞缪尔在1959年提出的（塞缪尔，1959年），是计算机科学的一个领域，它研究的是如何从数据中学习应用逻辑，而不是设计明确的算法。在设计明确算法不可行的计算任务中，机器学习最为成功，主要是在设计这类明确算法时缺乏足够的知识。也就是说，人们可以给出在不同情况下应该发生什么的例子，但不能为所有可能的输入制定通用的决策规则。例如，机器学习对自动语音识别、计算机视觉、自然语言处理和推荐系统等应用领域的近期改进做出了贡献。在这些应用领域，可以提供许多输入示例以及每个输入应该产生的结果，但没有算法能够正确处理所有可能的输入。

通过机器学习创建的应用逻辑可以根据它们执行的任务类型进行组织。机器学习任务的范围非常广泛。这里，我们展示一些任务中的几个：

- 1.分类：确定输入属于k个类别中的哪一个。一个例子是对象识别，比如确定照片中展示的是哪种食物。
- 2.回归：给定一些输入，预测一个数值。一个例子是预测下一个交易日结束时股票的价格。
- 3.转录：将非结构化数据转换为文本形式。一个例子是光学字符识别。
- 4.翻译：将一种语言中的象征序列转换为另一种语言的象征序列。一个例子是从英语翻译到中文。
- 5.嵌入：将输入转换为向量，同时保持实体之间的关系。一个例子是将自然语言句子转换为多维向量。

读者可以参考大量关于机器学习各种任务的数学背景和实际解决方案的文献。本章的目的是介绍神经网络方法在分类任务中涉及的计算核心。对这些核心的具体理解将使读者能够理解并为其其他机器学习任务开发深度学习方法的核心。因此，在本节中，我们将详细讨论分类任务，以建立理解神经网络所需的背景知识。从数学上讲，分类器是一个将输入映射到k个类别或标签的函数f：

$$f: \mathbb{R}^n \rightarrow \{1, 2, \dots, k\}$$

函数f由参数 θ 参数化，它将输入向量x映射到数值代码y，即，

$$y = f(x, \theta)$$

参数 θ 通常被称为模型。它包含了从数据中学习得到的权重。 θ 的定义最好用一个具体的例子来说明。让我们考虑一个名为感知机的线性分类器（罗森布拉特，1957年）： $y = \text{sign}(W \cdot x + b)$ ，其中W是与x长度相同的权重向量，b是偏置常数。符号函数如果输入为正，则返回值1；如果输入为0，则返回0；如果输入为负，则返回-1。也就是说，符号函数作为分类器激活，即完成输入值到三个类别的映射： $\{-1, 0, 1\}$ ；因此它通常被称为激活函数。激活函数在感知机的线性函数中引入了非线性。

在这种情况下，模型 θ 是向量 $\mathbf{W}$ 和常数 b 的组合。模型的结构是一个符号函数，其输入是输入 x 元素的线性表达式，其中系数是 $\mathbf{W}$ 中的元素，常数是 b 。

图16.1显示了一个感知机示例，其中每个输入是一个二维（2D）向量 (x_1, x_2) 。线性感知机的模型 θ 由权重向量 (w_1, w_2) 和偏置常数 b 组成。如图16.1所示，线性表达式 $w_1 x_1 + w_2 x_2 + b$ 在 x_1 - x_2 空间中定义了一条线，将空间分成两部分：一部分是所有点使表达式大于零，另一部分是所有点使表达式小于零。所有在线上点使表达式等于0。

从视觉上看，给定一组 (w_1, w_2) 和 b 值的组合，我们可以在 (x_1, x_2) 空间中画一条线，如图16.1所示。例如，对于一个模型是 $(w_1, w_2) = (2, 3)$ 且 $b = -6$ 的感知机，我们可以很容易地通过连接与 x_1 轴的两个交点 $(\frac{-b}{w_1}, 0) = (3, 0)$ 和 x_2 轴的交点 $(0, \frac{-b}{w_2}) = (0, 2)$ 来画一条线。这样画出的线对应于方程 $2x_1 + 3x_2 - 6 = 0$ 。通过这个图，我们可以很容易地可视化输入点的结果：任何在线上点（如图16.1中蓝色点所示）被分类为类别1，任何在线上点被分类为类别0，任何在线下点（如图16.1中橙色点所示）被分类为类别-1。

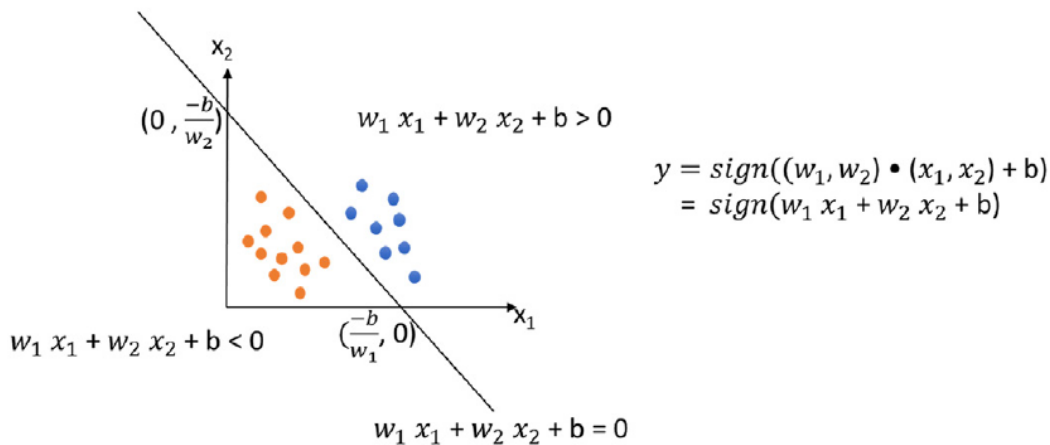


图16.1 输入是2维向量的感知机线性分类示例

计算输入的类别的过程通常被称为分类器的推理。在感知机的情况下，我们只需将输入坐标值代入 $y = \text{sign}(\mathbf{W} \cdot \mathbf{x} + b)$ 。以我们的例子为例，如果输入点是 $(5, -1)$ ，我们可以通过将它的坐标代入感知机函数来进行推理：

$$y = \text{sign}(2 * 5 + 3 * (-1) + 6) = \text{sign}(13) = 1$$

因此， $(5, -1)$ 被分类为类别1，即它位于蓝色点之中。

多层分类器

线性分类器在能够画出超平面（即在二维空间中的线和在三维空间中的平面）来划分空间并定义每个数据点类别的区域时非常有用。理想情况下，每个类别的数据点应该完全占据这样一个区域。例如，在二维的二类分类器中，我们需要能够画出一条线，将一类点与另一类点分开。不幸的是，这并不总是可行的。

考虑图16.2中的分类器。假设所有输入的坐标值都在 $[0,1]$ 的范围内。分类器应该将所有 x_1 和 x_2 值都大于0.5的点（落在域的右上角象限的点）分类为类别1，其余的分类为类别2。这个分类器可以用图16.2(A)中显示的线来大致实现。例如，一条 $2x_1 + 2x_2 - 3 = 0$ 的线可以正确分类大部分点。然而，一些 x_1 和 x_2 都大于0.5但总和小于1.5的橙色点，例如 $(0.55, 0.65)$ ，会被错误地分类为类别2（蓝色）。这是因为任何一条线都必然要么切掉右上角象限的一部分，要么包括域其余部分的一部分。没有一条单一的线能够正确分类所有可能的输入。

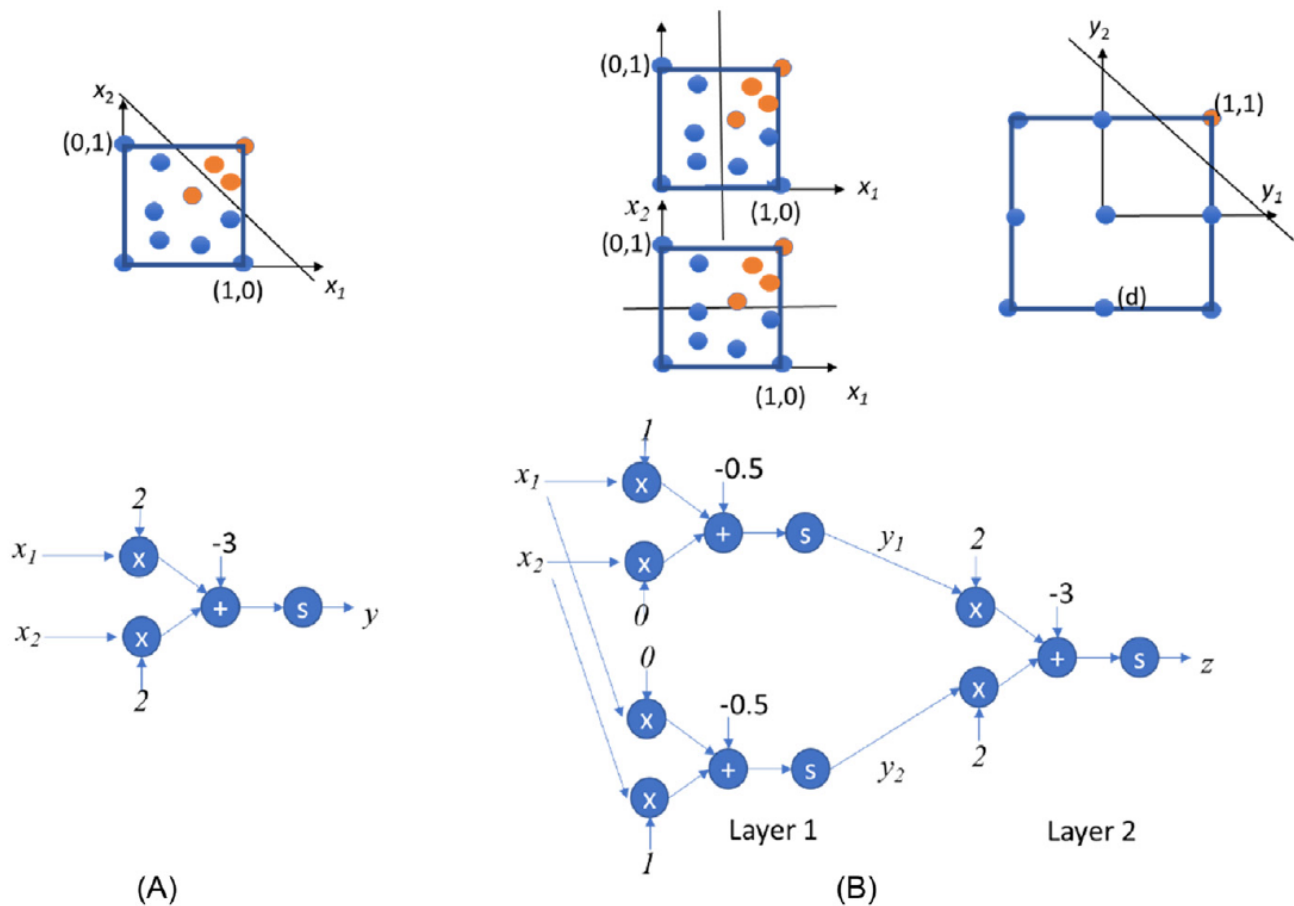


图16.2 多层感知机的例子

多层感知器 (MLP) 允许使用多条线来实现更复杂的分类模式。在多层感知器中，每一层由一个或多个感知器组成。一个层中感知器的输出是下一个层的输入。一个有趣且有用的属性是，尽管第一层的输入有无限多的可能值，但第一层的输出，从而第二层的输入，只能有数量有限的可能值。例如，如果图16.1中的感知器用作第一层，其输出将限制在 $\{-1, 0, 1\}$ 。

图16.2(B)显示了一个可以精确实现所需分类模式的双层感知器。第一层由两个感知器组成。第一个感知器， $y_1 = \text{sign}(x_1 - 0.5)$ ，将所有 x_1 坐标大于0.5的点分类为类别1；也就是说，输出值为1。其余的点被分类为类别-1或类别0。第一层的第二个分类器， $y_2 = \text{sign}(x_2 - 0.5)$ ，将所有 x_2 坐标大于0.5的点分类为类别1。其余的点被分类为类别-1。

因此，第一层的输出 (y_1, y_2) 只能是以下九种可能性之一： $(-1, -1)$, $(-1, 0)$, $(-1, 1)$, $(0, -1)$, $(0, 0)$, $(0, 1)$, $(1, -1)$, $(1, 0)$, $(1, 1)$ 。也就是说，只有九种可能的输入对值到第二层。在这些九种可能性中， $(1, 1)$ 是特殊的。所有原始输入点中的橙色类别都被第一层映射到 $(1, 1)$ 。因此我们可以使用第二层的一个简单感知器在 $(1, 1)$ 和 y_1 - y_2 空间中的其他八个可能点之间画一条线，如图16.2B所示。这可以通过一条 $2y_1 + 2y_2 - 3 = 0$ 的线或许多其他小变化的线来完成。

让我们使用在图16.2A中被单层感知器错误分类的 $(0.55, 0.65)$ 输入。当通过图16.2B中的双层感知器处理时，第一层的上层感知器生成 $y_1 = 1$ ；下层感知器生成 $y_2 = 1$ 。基于这些输入值，第二层的感知器生成 $z = 1$ ，这是 $(0.55, 0.65)$ 的正确分类。

请注意，双层感知器仍然有显著的限制。例如，假设我们需要构建一个感知器来分类图16.3A中显示的输入点。橙色输入点的值可以导致第二层的输入值 $(-1, -1)$ 或 $(1, 1)$ 。我们看到没有办法用一条线来正确分类第二层的点。我们在图16.2B中展示了通过在第二层添加另一个感知器来添加另一条线。函数将是 $z_2 = \text{sign}(2 * y_1 - 2 * y_2 - 3)$ 或其小变化。读者应该验证图16.3B中的所有蓝色点都将被映射到 z_1 - z_2 空间中的 $(-1, -1)$ 。而 y_1 - y_2 空间

中的(1, 1)和(-1, -1)被映射到 z_1 - z_2 空间中的(1, -1)和(-1, 1)。现在我们可以用一条线 $z_1 + z_2 + 1 = 0$ 或其小变化来正确分类点，如图16.3C所示。显然，如果我们需要将输入域划分为更多的区域，我们可能需要更多的层来进行正确的分类。

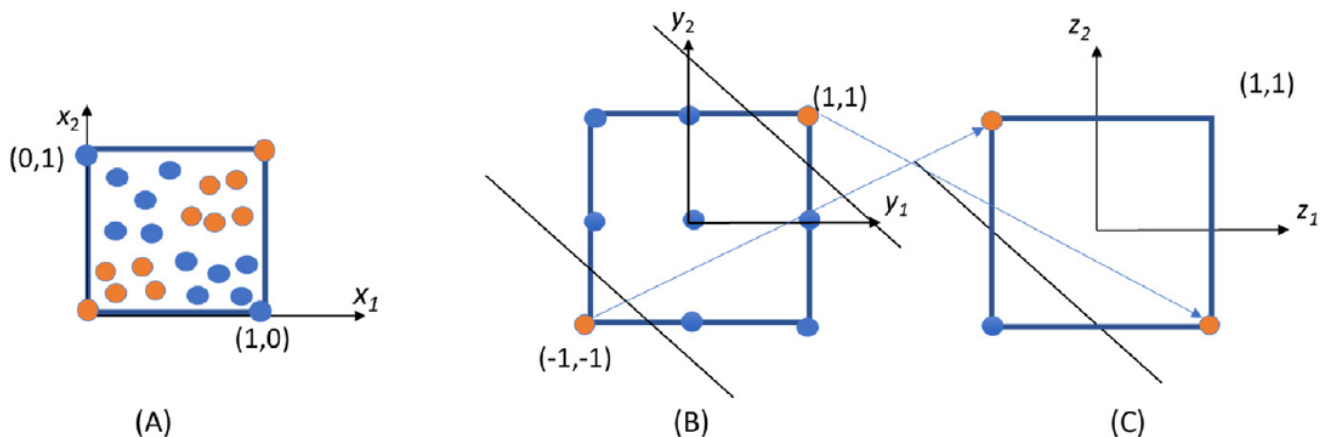


图16.3 需要多余两层感知机

图16.2B中的层1是一个全连接层的一个小例子，其中每个输出（即： $y_1; y_2$ ）是每个输入（即： $x_1; x_2$ ）的函数。通常，在全连接层中，每个 m 个输出都是所有 n 个输入的函数。全连接层的所有权重形成一个 $m \times n$ 的权重矩阵 W ，其中每行 m 是应用于输入向量（大小为 n 个元素）以产生 m 个输出之一的权重向量（大小为 n 个元素）。因此，评估全连接层的所有输出是从输入的矩阵-向量乘法。正如我们将看到的，全连接层是许多类型神经网络的核心组件，我们将进一步研究GPU实现。

当 m 和 n 变大时，全连接层变得非常昂贵。主要原因是全连接层需要一个 $m \times n$ 的权重矩阵。例如，在图像识别应用中， n 是输入图像中的像素数量， m 是对输入像素需要执行的分类数量。在这种情况下，对于高分辨率图像， n 是数百万，而 m 可能是数百或更多，这取决于需要识别的对象种类。此外，图像中的对象可能具有不同的比例和方向；可能需要许多分类器来处理这些变化。

将所有这些分类器的所有输入都提供给它们既昂贵又可能浪费。卷积层通过减少每个分类器接收的输入数量并跨分类器共享相同的权重来降低全连接层的成本。在卷积层中，每个分类器只接收输入图像的一个补丁，并根据权重对补丁中的像素执行卷积。输出称为输出特征图，因为输出中的每个像素都是分类器的激活结果。跨分类器共享权重允许卷积层有大量的分类器，即大 m 值，而没有过多的权重。在计算上，这可以作为2D卷积实现。然而，这种方法有效地将相同的分类器应用于图像的不同部分。我们可以对相同的输入应用不同的权重集，并生成多个输出特征图，正如我们稍后在本章中将看到的。

训练模型

到目前为止，我们假设分类器使用的模型参数是某种方式可用的。现在我们转向训练，或者说是使用数据来确定模型参数 θ 的值，包括权重($w_1; w_2$)和偏置 b 的过程。为了简单起见，我们将假设进行监督训练，其中使用带有期望输出值标记的输入数据来确定权重和偏置值。其他训练方式，如半监督和强化学习，也已经开发出来，以减少对标记数据的依赖。读者可以参考文献来理解在这些情况下如何完成训练。

误差函数

一般来说，训练将模型参数视为未知变量，并在给定标记的输入数据的情况下解决一个逆问题。在图16.1中的感知器示例中，每个用于训练的数据点都会被标记其期望的分类结果： -1 、 0 或 1 。训练过程通常从对($w_1; w_2$)和 b 值的初始猜测开始，并在输入数据上执行推理并生成分类结果。这些分类结果与标签进行比较。定义了一个误

差函数，有时也称为代价函数，以量化每个数据点的分类结果与相应标签之间的差异。例如，假设 y 是分类输出类别， t 是标签。以下是误差函数的一个示例：

$$E = \frac{(y-t)^2}{2}$$

这个误差函数具有一个很好的特性，即只要 y 和 t 的值之间有任何差异，无论是正的还是负的，误差值总是正的。如果我们需要对许多输入数据点的误差进行求和，正的和负的差异都将有助于总和，而不是相互抵消。也可以将误差定义为差异的绝对值，等等。正如我们将看到的，系数 $1/2$ 简化了解决模型参数所涉及的计算。

随机梯度下降

训练过程将尝试找到使所有训练数据点的误差函数值之和最小化的模型参数值。这可以通过随机梯度下降方法来完成，该方法反复运行输入数据集的不同排列通过分类器，发展参数值，并检查参数值是否已经收敛，即它们的值已经稳定并且自上次迭代以来变化小于阈值。一旦参数值收敛，训练过程就结束了。

周期(Epoch)

在训练过程的每次迭代中，称为一个周期，训练输入数据集首先被随机打乱，即排列，然后才被送入分类器。这种输入数据顺序的随机化有助于避免次优解。对于每个输入数据元素，将其分类器输出 y 值与标签数据进行比较以生成误差函数值。在我们的感知器示例中，如果数据标签是(类别)1，分类器输出是(类别)-1，使用 $E = (y - t)^2$ 的误差函数值将是2。如果误差函数值大于阈值，则激活反向传播操作以对参数进行更改，以便可以减少推理误差。

反向传播

反向传播的思想是从误差函数开始，回顾分类器并识别每个参数对误差函数值的贡献方式 (LeCun等人，1990)。如果一个参数的值增加导致数据元素的误差函数值增加，我们应该减少该参数的值，以便该数据点的误差函数值可以减少。否则，我们应该增加参数的值以减少该数据点的误差函数值。在数学上，一个函数的值随着其输入变量之一的变化而变化的速率和方向是该函数对该变量的偏导数。对于感知器，模型参数和输入数据点在计算误差函数的偏导数时被视为输入变量。因此，反向传播操作将需要派生出每个触发反向传播操作的输入数据元素对模型参数的误差函数的偏导数值。

让我们使用感知器 $y = \text{sign}(w_1 x_1 + w_2 x_2 + b)$ 来说明反向传播操作。假设误差函数 $E = (y - t)^2$ ，并且反向传播由训练输入数据元素 (5, 2) 触发。目标是修改 w_1 、 w_2 和 b 的值，以便感知器更有可能正确地对 (5, 2) 进行分类。也就是说，我们需要派生出偏导数 $\frac{\partial E}{\partial w_1}$ 、 $\frac{\partial E}{\partial w_2}$ 和 $\frac{\partial E}{\partial b}$ 的值，以便对 w_1 、 w_2 和 b 的值进行更改。

链式法则

我们可以看到 E 是 y 的函数，而 y 是 w_1 、 w_2 和 b 的函数。因此，我们可以使用链式法则来派生这些偏导数。对于 w_1 ,

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial w_1}$$

$\frac{\partial E}{\partial y}$ 是很简单的：

$$\frac{\partial E}{\partial y} = \frac{\partial \frac{(y-t)^2}{2}}{\partial y} = y-t$$

然而我们面临一个挑战：注意到符号函数不是一个可微函数，因为它在0处不连续。为了解决这个问题，机器学习界通常使用符号函数的平滑版本，该版本在0附近可微且在远离0的x值处接近符号函数的值。一个简单的平滑版本的例子是sigmoid函数 $s = \frac{1}{1 + e^{-x}}$ 。对于绝对值大的负x值，sigmoid表达式由 e^{-x} 项主导，sigmoid函数值将大约是 -1。对于绝对值大的正x值， e^{-x} 项减小，函数值将大约是1。对于接近0的x值，函数值从接近-1迅速增加到接近1。因此，sigmoid函数密切近似符号函数的行为，但对所有x值都是连续可微的。通过从符号函数改为sigmoid函数，感知器变为 $y = \text{sigmoid}(w_1 x_1 + w_2 x_2 + b)$ 。我们可以将 $\frac{\partial y}{\partial w_1}$ 表示为 $\frac{\partial \text{sigmoid}(k)}{\partial k} \cdot \frac{\partial k}{\partial w_1}$ ，使用中间变量 $k = w_1 x_1 + w_2 x_2 + b$ 。根据微积分操作， $\frac{\partial k}{\partial w_1}$ 简单地是 x_1 并且

$$\begin{aligned}\frac{\partial \text{sigmoid}(k)}{\partial k} &= \left(\frac{1 - e^{-k}}{1 + e^{-k}} \right)' = (1 - e^{-k})' \left(\frac{1}{1 + e^{-k}} \right) + (1 - e^{-k}) \left(\frac{1}{1 + e^{-k}} \right)', \\ &= e^{-k} \left(\frac{1}{1 + e^{-k}} \right) + (1 - e^{-k}) \left(-1 \times \left(\frac{1}{1 + e^{-k}} \right)^2 (-e^{-k}) \right) = \frac{2e^{-k}}{(1 + e^{-k})^2}\end{aligned}$$

把它们放到一起，我们有：

$$\frac{\partial E}{\partial w_1} = (y - t) \frac{2e^{-k}}{(1 + e^{-k})^2} x_1$$

类似的：

$$\begin{aligned}\frac{\partial E}{\partial w_2} &= (y - t) \frac{2e^{-k}}{(1 + e^{-k})^2} x_2 \\ \frac{\partial E}{\partial b} &= (y - t) \frac{2e^{-k}}{(1 + e^{-k})^2}\end{aligned}$$

其中：

$k = w_1 x_1 + w_2 x_2 + b$ 应该清楚的是，所有三个偏导数的值都可以通过输入数据 (x_1 、 x_2 和 t) 和当前模型参数值 (w_1 、 w_2 和 b) 的组合完全确定。反向传播的最后一步是修改参数值。回想一下，一个函数对一个变量的偏导数给出了当变量改变其值时函数值变化的方向和速率。如果误差函数对一个参数的偏导数在给定输入数据和当前参数值的组合下有一个正值，我们想要减少该参数的值，以便误差函数值会减少。另一方面，如果误差函数对变量的偏导数有一个负值，我们想要增加该参数的值，以便误差函数值会减少。

学习率

在数值上，我们希望对那些对误差函数变化更敏感的参数进行更大的更改，也就是说，当误差函数相对于该参数的偏导数的绝对值为较大值时。这些考虑使我们从每个参数中减去一个与该参数的误差函数偏导数成比例的值。这是通过将偏导数乘以一个常数 ϵ 来实现的，在机器学习中称为学习率常数，然后再从参数值中减去。 ϵ 越大，参数值的演变就越快，因此可能可以在较少的迭代次数内达到解决方案。然而，较大的 ϵ 也增加了不稳定性，并阻止参数值收敛到解决方案。在我们的感知器示例中，参数的修改如下：

$$w_1 \leftarrow w_1 - \varepsilon \frac{\partial E}{\partial w_1}$$

$$w_2 \leftarrow w_2 - \varepsilon \frac{\partial E}{\partial w_2}$$

$$b \leftarrow b - \varepsilon \frac{\partial E}{\partial b}$$

在本章的其余部分中，我们将使用一个通用符号 θ 来表示公式和表达式中的模型参数。也就是说，我们将用一个通用表达式来表示上面的三个表达式：

$$\theta \leftarrow \theta - \varepsilon \frac{\partial E}{\partial \theta}$$

读者应该理解，对于这些通用表达式中的每一个，都可以用任何参数替换 θ ，以将表达式应用于该参数。

Minibatch

在实践中，由于反向传播过程相当昂贵，它不是由推理结果与标签不同的单个数据点触发的。相反，在每个周期中输入被随机打乱后，它们被分成称为minibatch（小批量）的段。训练过程通过整个minibatch进行推理，并累积它们的误差函数值。如果minibatch中的总误差过大，就会为该minibatch触发反向传播。在反向传播期间，检查minibatch中每个数据点的推理结果，如果不正确，就使用该数据来派生偏导数值，如上所述用来修改模型参数值。

训练多层分类器

对于多层分类器，反向传播从最后一层开始，并如上所述修改该层的参数值。问题是，我们应该如何修改前一层参数。请记住，我们可以基于 $\frac{\partial E}{\partial y}$ 派生 $\frac{\partial E}{\partial \theta}$ ，正如我们对最后一层所演示的。一旦我们有了前一层的 $\frac{\partial E}{\partial y}$ ，我们就有了计算该层参数修改所需的一切。

一个简单但重要的观察是，前一层的输出 y 也是最后一层的输入。因此，前一层的 $\frac{\partial E}{\partial y}$ 实际上就是最后一层的 $\frac{\partial E}{\partial \theta}$ 。因此，关键是在我们修改最后一层的参数值后，派生出 $\frac{\partial E}{\partial x}$ 。正如我们下面看到的， $\frac{\partial E}{\partial x}$ 与 $\frac{\partial E}{\partial \theta}$ 并没有太大不同，即 $\frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial x}$ 。

$\frac{\partial y}{\partial x}$ 也相当直接，因为输入在 y 方面的作用与参数相同。我们只需要对中间函数 k 对输入进行偏导数。对于我们的感知器示例，我们有

$$\frac{\partial E}{\partial x_1} = (y - t) \frac{2e^{-k}}{(1 + e^{-k})^2} w_1$$

$$\frac{\partial E}{\partial x_2} = (y - t) \frac{2e^{-k}}{(1 + e^{-k})^2} w_2$$

其中 $k = w_1 x_1 + w_2 x_2 + b$ 。在图16.2B中的感知器示例中，最后一层（第2层）的 x_1 是第1层顶部感知器的输出 y_1 ， x_2 是第1层底部感知器的输出 y_2 。现在我们准备开始计算前一层两个感知器的 $\frac{\partial E}{\partial \theta}$ 。显然，如果有更多层，这个过程可以重复进行。

前馈网络

通过将多层分类器连接起来，并将每一层的输出提供给下一层，我们形成了一个前馈网络。图16.2B展示了一个双层前馈网络的例子。我们对多层感知器（MLP）进行推理和训练的所有讨论都假设了这一特性。在前馈网络中，较早层的所有输出都会传送到一个或多个较晚层。没有从较晚层的输出到较早层输入的连接。因此，反向传播可以简单地从最后阶段开始向后迭代，不会产生由反馈循环引起的复杂性。

16.2 卷积神经网络

深度学习过程（LeCun等人，2015）使用一系列特征提取器来学习复杂特征，如果有足够多的训练数据允许系统适当地训练所有层级特征提取器的参数，以自动发现足够数量的相关模式，那么它可以取得更准确的模式识别结果。有一种类别的深度学习过程比其他过程更容易训练，并且具有更好的泛化能力。这些深度学习过程基于一种特殊类型的前馈网络，称为卷积神经网络（CNN）。

CNN是在20世纪80年代末发明的（LeCun等人，1998）。到了20世纪90年代初，CNN已经被应用于自动语音识别、光学字符识别（OCR）、手写识别和面部识别（LeCun等人，1990）。然而，直到20世纪90年代末，计算机视觉和自动语音识别的主流仍然基于精心设计的特征。标记数据的数量不足以使深度学习系统与人类专家构建的识别或分类功能竞争。人们普遍认为，自动构建具有足够层数的分层特征提取器，在计算上是不可行的，这些层数足以比人类定义的特定应用特征提取器表现得更好。

2006年左右，一群研究人员通过引入一种无监督学习方法重新点燃了对深度前馈网络的兴趣，这种学习方法可以在不需要标记数据的情况下创建多层、分层特征检测器（Hinton等人，2006；Raina等人，2009）。这种方法的第一个重大应用是在语音识别领域。这一突破得以实现的原因是GPU，它使研究人员能够以比传统CPU快十倍的速度训练网络。这一进步，加上在线可用的大量媒体数据，极大地提升了深度学习方法的地位。尽管它们在语音识别方面取得了成功，但CNN在计算机视觉领域直到2012年基本上被忽视了。

2012年，一组来自多伦多大学的研究人员训练了一个大型、深度卷积神经网络，用于在ILSVRC比赛中对1000个不同的类别进行分类（Krizhevsky等人，2012）。按照当时的标准，这个网络是巨大的：它大约有6000万个参数和650,000个神经元。它在ImageNet数据库的120万张高分辨率图像上进行了训练。该网络仅在两周内在两个GPU上使用基于CUDA的卷积神经网络库进行训练，该库由Alex Krizhevsky编写（Krizhevsky）。该网络取得了突破性的结果，以15.3%的获胜测试错误率获胜。相比之下，使用传统计算机视觉算法的第二名团队的错误率为26.2%。这一成功引发了计算机视觉的革命，CNN成为了计算机视觉、自然语言处理、强化学习以及许多其他传统机器学习领域的主流工具。

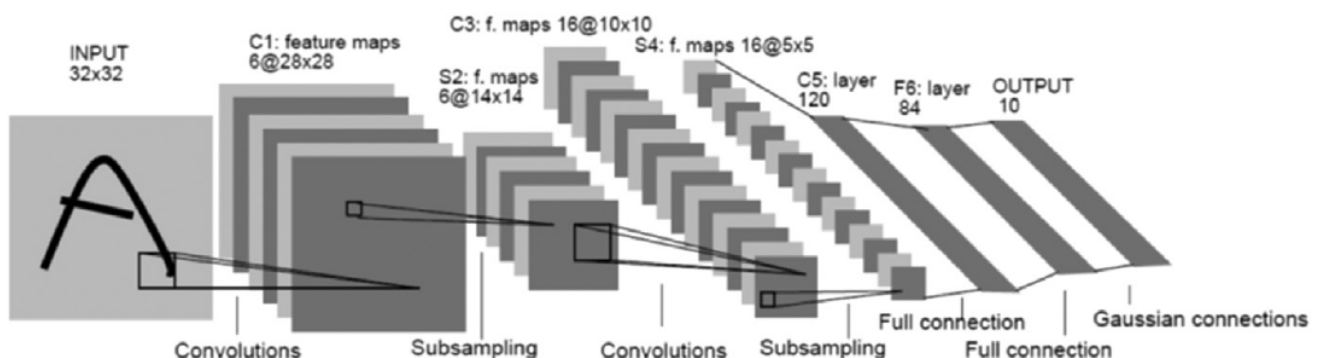


图16.4 LeNet-5，一种用于手写数字识别的卷积神经网络。输入中的字母A应该被分类为十个类别（数字）之

外。

本节介绍了CNN推理和训练的顺序实现。我们将使用LeNet-5，这是一种在20世纪80年代末为数字识别而设计的网络（LeCun等人，1990）。如图16.4所示，LeNet-5由三种类型的层组成：卷积层、池化层和全连接层。这三种类型的层继续是当今神经网络的关键组成部分。我们将考虑每种类型的层的逻辑设计和顺序实现。网络的输入显示为灰度图像，手写数字表示为2D 32x32像素数组。最后一层计算输出，即原始图像属于网络设置识别的十个类别（数字）中每一个的概率。

卷积神经网络推理

卷积网络中的计算被组织成一系列层的顺序。我们将层的输入和输出称为特征图或简称为特征。例如，在图16.4中，网络输入端的C1卷积层的计算被组织成从INPUT像素数组生成六个输出特征图。输入特征图要产生的输出由像素组成，每个像素的产生是通过执行前一层产生的特征图像素（C1的情况下是INPUT）的一个小局部补丁和一组权重（即卷积滤波器，如第七章：卷积中定义）所称的滤波器组之间的卷积。然后，卷积结果被送入激活函数（如sigmoid）以产生输出特征图中的一个输出像素。可以认为每个输出特征图中的卷积层对于每个像素来说是一个感知器，其输入是输入特征图中的像素补丁。也就是说，每个输出像素的值是来自所有输入特征图中相应补丁的卷积结果之和。

图16.5展示了一个小卷积层的例子。有三个输入特征图、两个输出特征图和六个滤波器组。层中不同的输入和输出特征图对使用不同的滤波器组。由于图16.5中有三对输入和输出特征图，我们需要 $3 \times 3 \times 2 = 6$ 个滤波器组。在图16.4中LeNet的C3层有六个输入特征图和16个输出特征图。因此C3中使用了总共 $6 \times 3 \times 16 = 96$ 个滤波器组。

图16.5B更详细地说明了卷积层所做的计算。为了简化，我们省略了输出像素的激活函数。我们展示了每个输出特征图是所有输入特征图的卷积之和。例如，输出特征图0的左上角元素（值14）是通过对输入特征图的圈出补丁和相应的滤波器组进行卷积来计算的：

$$\begin{aligned} & (1, 2, 1, 1) \cdot (1, 1, 2, 2) + (0, 2, 0, 3) \cdot (1, 1, 1, 1) + (1, 2, 0, 1) \cdot (0, 1, 1, 0) \\ &= 1 + 2 + 2 + 2 + 0 + 2 + 0 + 3 + 0 + 2 + 0 + 0 \\ &= 14 \end{aligned}$$

也可以将三个输入图视为一个3D输入特征图，将三个滤波器组视为一个3D滤波器组。每个输出特征图简单地是3D输入特征图和3D滤波器组的3D卷积结果。在图16.5B中，左侧的三个2D滤波器组形成一个3D滤波器组，右侧的三个形成第二个3D滤波器组。一般来说，如果一个卷积层有n个输入特征图和m个输出特征图，就会使用 $n \times m$ 个不同的2D滤波器组。也可以将这些滤波器组视为m个3D滤波器组。尽管图16.4中没有显示，LeNet-5中使用的所有2D滤波器组都是 5×5 的卷积滤波器。

回想第七章，卷积，从输入图像和卷积滤波器生成卷积输出图像需要对“幽灵单元”做出假设。与其做出这样的假设，LeNet-5的设计简单地使用每个维度边缘的两个元素作为幽灵单元。这将每个维度的大小减少了四：顶部两个、底部两个、左侧两个和右侧两个。因此，我们看到对于C1层，32x32的INPUT图像产生的输出特征图是一个28x28的图像。

图16.4通过展示C1层的一个像素是如何从INPUT像素的一个正方形（ 5×5 ，虽然没有明确显示）补丁生成的，来说明这个计算。

我们假设输入特征图存储在一个三维数组 $X[C, H, W]$ 中，其中C是输入特征图的数量，H是每个输入图图像的高度，W是每个输入图图像的宽度。也就是说，最高维度的索引选择其中一个特征图（通常称为通道），而较低两个维度的索引选择一个输入特征图中的一个像素。例如，C1层的输入特征图存储在 $X[1, 32, 32]$ 中，因为只有一个输入特征图（图16.4中的INPUT），它由x和y维度上各32个像素组成。这也反映了人们可以将层的2D输入特征图视为形成一个3D输入特征图。

卷积层的输出特征图也存储在一个三维数组 $Y[M, H - K + 1, W - K + 1]$ 中，其中 M 是输出特征图的数量， K 是每个2D滤波器的高度（和宽度）。例如，C1层的输出特征图存储在 $Y[6, 28, 28]$ 中，因为C1使用 5×5 滤波器生成六个输出特征图。滤波器组存储在一个四维数组 $W[M, C, K, K]$ 中。有 $M \times C$ 个滤波器组。当使用输入特征图 $X[c, ,]$ 计算输出特征图 $Y[m, ,]$ 时，使用滤波器组 $W[m, c, ,]$ 。回想一下，每个输出特征图是所有输入特征图的卷积之和。因此，我们可以将卷积层的前向传播路径视为一组 M 个3D卷积，其中每个3D卷积由一个3D滤波器组指定，该滤波器组是 W 的一个 $C \times K \times K$ 子矩阵。

图16.6展示了卷积层前向传播路径的顺序C实现。最外层（ m ）for循环（第04-12行）的每次迭代生成一个输出特征图。下两层（ h 和 w ）的for循环（第05-12行）的每次迭代生成当前输出特征图的一个像素。最内层三个循环（第08-11行）执行输入特征图和3D滤波器组之间的3D卷积。

```
01 void convLayer_forward(int M, int C, int H, int W, int K, float* X, float* W,
    float* Y) {
02     int H_out = H - K + 1;
03     int W_out = W - K + 1;

04     for(int m = 0; m < M; m++)          // for each output feature map
05         for(int h = 0; h < H_out; h++)    // for each output element
06             for(int w = 0; w < W_out; w++) {
07                 Y[m, h, w] = 0;
08                 for(int c = 0; c < C; c++) // sum over all input feature maps
09                     for(int p = 0; p < K; p++) // KxK filter
10                         for(int q = 0; q < K; q++)
11                             Y[m, h, w] += X[c, h + p, w + q] * W[m, c, p, q];
12             }
13 }
```

图16.6 卷积层前向传播路径的C语言实现。

卷积层的输出特征图通常经过一个池化层（也称为子采样层）。池化层通过组合像素来减小图像图的大小。例如，在图16.4中，池化层S2接收六个大小为 28×28 的输入特征图，并生成六个大小为 14×14 的特征图。池化输出特征图中的每个像素都是从相应输入特征图中的 2×2 邻域生成的。这四个像素的值被平均以形成一个输出特征图中的像素。池化层的输出与前一具有相同数量的输出特征图，但每个图的行数和列数减半。例如，池化层S2的输出特征图数量（六个）与其输入特征图的数量相同，或与C1卷积层的输出特征图数量相同。

图16.7展示了池化层前向传播路径的顺序C实现。最外层（ m ）for循环（第02-11行）的每次迭代生成一个输出特征图。接下来的两层（ h 和 w ）的for循环（第03-11行）生成当前输出图中的单个像素。两个最内层的for循环（第06-09行）在邻域中求和。在我们的LeNet-5池化示例中， K 等于2。然后向每个输出特征图中的每个输出像素添加特定于该输出特征图的偏置值 $b[m]$ ，并将总和通过sigmoid激活函数。读者应该认识到，每个输出像素是通过相当于一个感知器生成的，该感知器将每个特征图中的四个输入像素作为其输入，并在相应的输出特征图中生成一个像素。ReLU是另一种常用的激活函数，它是一个简单的非线性滤波器，只传递非负值：如果 $X \geq 0$ ，则 $Y = X$ ，否则为0。

```

01 void subsamplingLayer_forward(int M, int H, int W, int K, float* Y, float*
S){
02     for(int m = 0; m < M; m++)                // for each output feature map
03         for(int h = 0; h < H/K; h++)            // for each output element,
04             for(int w = 0; w < W/K; w++) {        // this code assumes that H and W
05                 S[m, x, y] = 0.;                // are multiples of K
06                 for(int p = 0; p < K; p++) {        // loop over KxK input samples
07                     for(int q = 0; q < K; q++)
08                         S[m, h, w] += Y[m, K*h + p, K*w + q] / (K*K);
09                 }
10                 // add bias and apply non-linear activation
11                 S[m, h, w] = sigmoid(S[m, h, w] + b[m]);
12     }

```

图16.7 池化层前向传播路径的顺序C语言实现。该层还包括一个激活函数，该函数被包含在内。

为了完成我们的例子，卷积层C3有16个输出特征图，每个特征图是10x10的图像。这层有 $6 \times 16 = 96$ 个滤波器组，每个滤波器组有 $5 \times 5 = 25$ 个权重。C3的输出被送入池化层S4，它生成了16个5x5的输出特征图。最后，最后一个卷积层C5使用 $16 \times 120 = 1920$ 个5x5滤波器组从其16个输入特征图中生成120个单像素输出特征。

这些特征图通过全连接层F6传递，该层有84个输出单元，其中每个输出都与所有输入完全连接。输出是权重矩阵 W 与输入向量 X 的乘积，然后添加偏置，并通过sigmoid传递。对于F6示例， W 是120x84的矩阵。总之，输出是一个84元素的向量 $Y_6 = \text{sigmoid}(W X + b)$ 。读者应该认识到，这相当于84个感知器，每个感知器将C5层生成的所有120个单像素 x 值作为其输入。我们把全连接层的详细实现作为练习留给读者。

最后一个阶段是一个输出层，它使用高斯滤波器生成一个包含十个元素的向量，这些元素对应于输入图像包含十个数字之一的概率。

卷积神经网络反向传播

CNN的训练基于我们在第16.1节讨论过的随机梯度下降方法和反向传播过程（Rumelhart等人，1986）。训练数据集被标记有“正确答案”。在手写识别示例中，标签给出了图像中正确的字母。标签信息可以用来生成最后阶段的“正确”输出：正确的十元素向量的概率值，其中正确数字的概率为1.0，其他所有数字的概率为0.0。

对于每张训练图像，网络的最后阶段计算损失（错误）函数，作为生成的输出概率向量元素值和“正确”输出向量元素值之间的差异。给定一系列训练图像，我们可以数值计算损失函数相对于输出向量元素的梯度。直观地说，它给出了当输出向量元素的值改变时损失函数值变化的速率。

反向传播过程从计算最后一层的损失函数梯度 $\frac{\partial E}{\partial y}$ 开始。然后，它将梯度从最后一层向第一层通过网络的所有层传播。每一层接收到作为其输入的输出特征图的梯度 $\frac{\partial E}{\partial x}$ （这只是后续层的 $\frac{\partial E}{\partial x}$ ），并计算自己的梯度 $\frac{\partial E}{\partial x}$ ，如图16.8B所示。这个过程重复进行，直到完成调整网络的输入层。

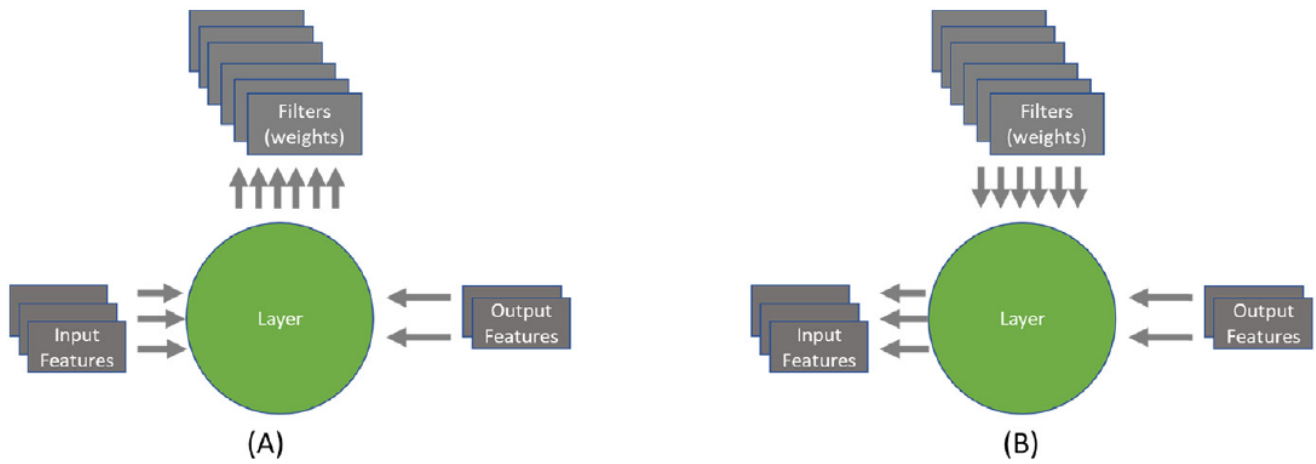


图16.8 在CNN中，某层的(A) $\frac{\partial E}{\partial w}$ 和(B) $\frac{\partial E}{\partial x}$ 的反向传播。

如果一个层学习到了参数（“权重”） w ，那么这层也计算它的损失相对于它的权重的梯度 $\frac{\partial E}{\partial w}$ ，如图16.8A所示。例如，全连接层给定为 $y = w \cdot x$ 。梯度的反向传播由以下方程给出：

$$\frac{\partial E}{\partial x} = w^T \frac{\partial E}{\partial y} \quad \text{and} \quad \frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} x^T$$

这个方程可以在元素的基础上推导，就像我们对双层感知器示例所做的那样。回想一下，每个全连接层输出像素是通过一个感知器计算的，它以输入特征图中的像素作为输入。正如我们在第16.1节训练MLP时所示，对于一个输入 x ， $\frac{\partial E}{\partial x}$ 是每个输出 y 元素对输入 x 元素的贡献和通过 w 值对 y 值的贡献的乘积之和。因为 w 矩阵的每一行都将所有的 x 元素（列）关联到全连接层的 y 元素（一行），所以 w 的每一列（即 w^T 的行）将所有的 y （即 $\frac{\partial E}{\partial y}$ ）元素关联回一个 x （即 $\frac{\partial E}{\partial x}$ ）元素，因为转置交换了行和列的角色。因此，矩阵-向量乘法 $w^T \frac{\partial E}{\partial y}$ 的结果是一个向量，它所有输入 x 元素的 $\frac{\partial E}{\partial x}$ 值。

类似地，由于每个 w 元素都乘以一个 x 元素来生成一个 y 元素，每个 w 元素的 $\frac{\partial E}{\partial w}$ 可以被计算为一个元素 $\frac{\partial E}{\partial y}$ 和 x 元素的乘积。因此，（一个单列矩阵）和 x^T （一个单行矩阵）之间的矩阵乘法结果是一个全连接层所有 w 元素的 $\frac{\partial E}{\partial w}$ 值的矩阵。这也可以被看作是 $\frac{\partial E}{\partial y}$ 和 x 向量之间的外积。

让我们将注意力转向卷积层的反向传播。我们将从 $\frac{\partial E}{\partial y}$ 计算 $\frac{\partial E}{\partial x}$ 开始，这最终将被用来计算前一层的梯度。输入 x 的通道 c 的梯度被给定所有 m 层输出的“反向卷积”与相应的 $W(m, c)$ 的总和：

$$\frac{\partial E}{\partial x}(c, h, w) = \sum_{m=0}^{M-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} \left(\frac{\partial E}{\partial y}(m, h-p, w-q) * w(m, c, k-p, k-q) \right)$$

反向卷积通过 $h-p$ 和 $w-q$ 的索引允许所有在前向卷积中从 x 元素获得贡献的输出 y 元素的梯度，通过相同的权重，为该 x 元素的梯度做出贡献。这是因为在卷积层的前向推理中， x 元素值的任何变化都会乘以这些 w 元素，并通过这些 y 元素对损失函数值的变化做出贡献。图16.9展示了使用 $3 \times 3 \times 3$ 滤波器组的一个小例子的索引模式。输出特征图中的九个阴影 y 元素是在前向干扰中从 $x_{\{h,w\}}$ 获得贡献的 y 元素。例如，输入元素 $x_{\{h,w\}}$ 通过与 $w_{\{2,2\}}$ 的乘法贡献于 $y_{\{h-2,w-2\}}$ ，并通过与 $w_{\{0,0\}}$ 的乘法贡献于 $y_{\{h,w\}}$ 。因此，在反向传播期间， $\frac{\partial E}{\partial x_{\{h,w\}}}$ 应该从这九个 $\frac{\partial E}{\partial y}$ 元素的值中获得贡献，计算等同于使用转置的滤波器组 $w_{\{T\}}$ 进行卷积。

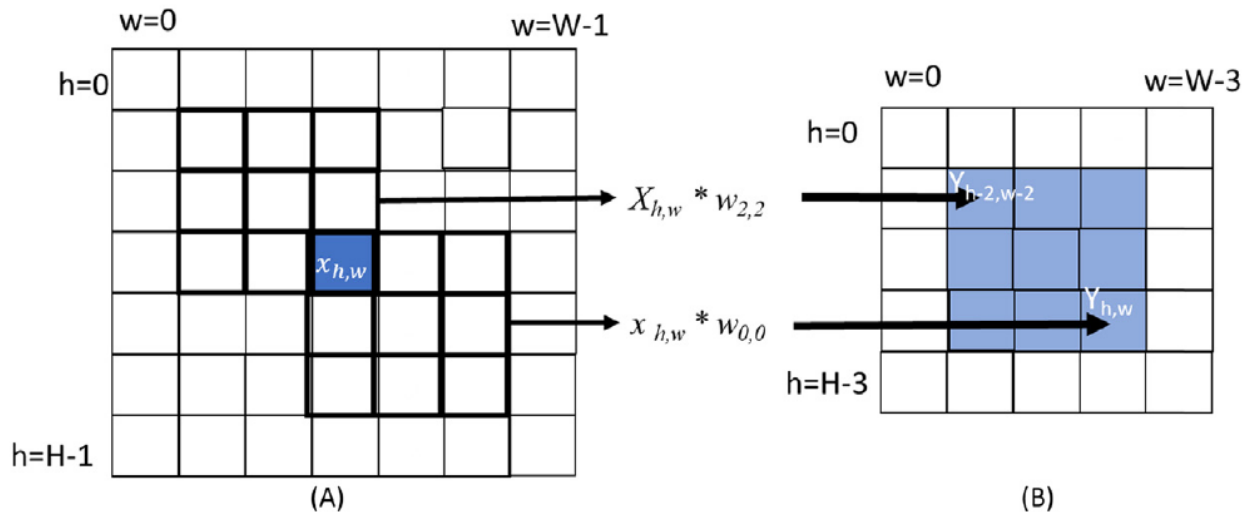


图16.9 卷积层。(A) $\frac{\partial E}{\partial w}$ 和 (B) $\frac{\partial E}{\partial x}$ 的反向传播。

图16.10展示了为每个输入特征图计算每个元素 $\frac{\partial E}{\partial x}$ 的C代码。注意，代码假设所有输出特征图的 $\frac{\partial E}{\partial y}$ 已经被计算出来，并通过指针参数 `dE_dY` 传入。这是合理的假设，因为对于当前层来说，其直接下一层的梯度在达到当前层之前应该已经在反向传播中计算过了。它还假设 $\frac{\partial E}{\partial x}$ 的空间已经在设备内存中分配，其句柄作为指针参数 `dE_dX` 传入。该函数生成所有元素的 $\frac{\partial E}{\partial x}$ 。

```

01 void convLayer_backward_x_grad(int M, int C, int H_in, int W_in, int K,
                                float* dE_dY, float* W, float* dE_dX) {
02     int H_out = H_in - K + 1;
03     int W_out = W_in - K + 1;
04     for(int c = 0; c < C; c++)
05         for(int h = 0; h < H_in; h++)
06             for(int w = 0; w < W_in; w++)
07                 dE_dX[c, h, w] = 0;

08     for(int m = 0; m < M; m++)
09         for(int h = 0; h < H-1; h++)
10             for(int w = 0; w < W-1; w++)
11                 for(int c = 0; c < C; c++)
12                     for(int p = 0; p < K; p++)
13                         for(int q = 0; q < K; q++)
14                             if(h-p >= 0 && w-p >= 0 && h-p < H_out and w-p < W_OUT)
15                                 dE_dX[c, h, w] += dE_dY[m, h-p, w-p] * W[m, c, k-p, k-q];
16 }

```

图16.10 卷积层反向路径的 $\frac{\partial E}{\partial x}$ 计算。

```

01 void convLayer_backward_w_grad(int M, int C, int H, int W, int K, float*
                                dE_dY, float* X, float* dE_dW) {
02     int H_out = H - K + 1;
03     int W_out = W - K + 1;
04     for(int m = 0; m < M; m++)
05         for(int c = 0; c < C; c++)
06             for(int p = 0; p < K; p++)
07                 for(int q = 0; q < K; q++)
08                     dE_dW[m, c, p, q] = 0.;
09
10     for(int m = 0; m < M; m++)
11         for(int h = 0; h < H_out; h++)
12             for(int w = 0; w < W_out; w++)
13                 for(int c = 0; c < C; c++)
14                     for(int p = 0; p < K; p++)
15                         for(int q = 0; q < K; q++)
16                             dE_dW[m, c, p, q] += X[c, h+p, w+q] * dE_dY[m, c, h, w];
17 }

```

图16.11 卷积层反向路径的 $\frac{\partial E}{\partial w}$ 计算。

卷积层计算中用于计算 $\frac{\partial E}{\partial w}$ 的顺序代码与 $\frac{\partial E}{\partial x}$ 的类似，如图16.11所示。由于每个 $W(m, c)$ 影响输出 $Y(m)$ 的所有元素，我们应该在相应的输出特征图中的所有像素上累积每个 $W(m, c)$ 的梯度：

$$\frac{\partial E}{\partial W}(m, c, p, q) = \sum_{h=0}^{H_{out}-1} \sum_{w=0}^{W_{out}-1} \left(X(c, h+p, w+q) * \frac{\partial E}{\partial Y}(m, h, w) \right)$$

请注意，虽然计算 $\frac{\partial E}{\partial x}$ 对于将梯度传播到前一层很重要，但计算 $\frac{\partial E}{\partial w}$ 是当前层权重值调整的关键。

在所有滤波器组元素位置的值都已计算后，使用第16.1节中介绍的公式更新权重，以最小化预期误差： $w \leftarrow w - \epsilon \frac{\partial E}{\partial w}$ ，其中 ϵ 是学习率常数。 ϵ 的初始值是经验设定的，并且根据用户定义的规则在周期中减少。 ϵ 的值在周期中减少，以确保权重收敛到最小误差。回想一下，调整项的负号导致变化方向与梯度方向相反，这样变化可能会减少误差。还记得，层的权重值决定了输入是如何通过网络转换的。所有层的这些权重值的调整适应了网络的行为。也就是说，网络“学习”了标记的训练数据序列，并通过调整所有层的所有权重值来适应其行为，这些调整是针对那些推理结果不正确并触发了反向传播的输入的。

正如我们在第16.1节中讨论的，反向传播通常在对训练数据集中的N幅图像的minibatch进行前向传递后触发，并且已经为这个minibatch计算了梯度。学习到的权重会根据为minibatch计算的梯度进行更新，然后用另一个minibatch重复该过程。这之前描述的所有数组添加了一个额外的维度，用n作为minibatch中样本的索引。它还在样本上增加了一个额外的循环。

图16.12展示了卷积层前向路径实现的修订版。它为minibatch的所有样本生成输出特征图。

```

01 void convLayer_batched(int N, int M, int C, int H, int W, int K, float* X,
                        float* W, float* Y) {
02     int H_out = H - K + 1;
03     int W_out = W - K + 1;
04     for(int n = 0; n < N; n++)          // for each sample in the mini-batch
05         for(int m = 0; m < M; m++)      // for each output feature map
06             for(int h = 0; h < H_out; h++) // for each output element
07                 for(int w = 0; w < W_out; w++) {
08                     Y[n, m, h, w] = 0;
09                     for (int c = 0; c < C; c++ // sum over all input feature maps
10                         for (int p = 0; p < K; p++)          // KxK filter
11                             for (int q = 0; q < K; q++)
12                                 Y[n,m,h,w] = Y[n,m,h,w] + X[n, c, h+p, w+q]*W[m,c,p,q];
13                 }
14     }

```

图16.12 带小批量训练的卷积层的前向路径。

16.3 卷积层：一个CUDA推理内核

在训练卷积神经网络时的计算模式类似于矩阵乘法：它既计算密集又高度并行。我们可以并行处理minibatch中的不同样本、相同样本的不同输出特征图以及每个输出特征图中的不同元素。在图16.12中， n 循环（第04行，遍历minibatch中的样本）、 m 循环（第05行，遍历输出特征图）以及嵌套的 h - w 循环（第06-07行，遍历每个输出特征图的像素）都是可以并行执行的并行循环。这四个循环级别共同提供了巨大的并行度。

最内层的三个循环级别， c 循环（遍历输入特征图或通道）和嵌套的 p - q 循环（遍历滤波器组中的权重），也提供了相当程度的并行性。然而，要并行化它们，我们需要在累积到 Y 元素时使用原子操作，因为这些循环级别的不同迭代可能会对相同的 Y 元素执行读写修改。因此，除非我们真的需要更多的并行性，否则我们将保持这些循环的串行化。

假设我们利用卷积层中四个“容易”的并行级别（ n 、 m 、 h 、 w ），总的并行迭代次数是 $N \times M \times H_{\text{out}} \times W_{\text{out}}$ 的乘积。这种高度可用的并行性使卷积层成为GPU加速的极佳候选。我们可以很容易地设计一个线程组织，以捕获并行性。

我们首先需要对线程组织做一些高层设计决策。假设我们将让每个线程计算一个输出特征图的一个元素。我们使用2D线程块，在这些线程块中，每个线程块计算一个输出特征图中 $\text{TILE_WIDTH} \times \text{TILE_WIDTH}$ 像素的瓦片。例如，如果我们设置 $\text{TILE_WIDTH} = 16$ ，每个块将总共有256个线程。这在处理每个输出特征图的像素时捕获了嵌套的 h - w 循环级别的部分并行性。

块可以以几种不同的方式组织成3D网格。每种选项都以不同的组合指定网格维度，以捕获 n 、 m 和 h - w 并行性。我们将详细介绍其中一个选项，并将其作为练习留给读者，以探索不同选项并评估每种选项的潜在利弊。我们详细介绍的选项如下：

1. 第一维（ X ）对应于每个块覆盖的（ M ）输出特征图。
2. 第二维（ Y ）反映了块的输出瓦片在输出特征图中的位置。
3. 网格中的第三维（ Z ）对应于minibatch中的样本（ N ）。

```

01 # define TILE_WIDTH 16
02 W_grid = W_out/TILE_WIDTH; // number of horizontal tiles per output map
03 H_grid = H_out/TILE_WIDTH; // number of vertical tiles per output map
04 T = H_grid * W_grid;
05 dim3 blockDim(TILE_WIDTH, TILE_WIDTH, 1);
06 dim3 gridDim(M, T, N);
07 ConvLayerForward Kernel<<< gridDim, blockDim>>> (...);

```

图16.13 主机代码用于启动卷积层内核。

图16.13展示了基于上述线程组织启动内核的主机代码。网格的X和Z维度的块数是直接的；它们简单地是输出特征图的数量M和minibatch中的样本数量N。Y维度的排列稍微复杂一些，如图16.14所示。理想情况下，我们希望将网格索引的两个维度分别用于垂直和水平瓦片索引，以简化问题。然而，我们只有两个维度，因为我们在X上使用输出特征图索引，在Z上使用minibatch中的样本索引。因此，我们线性化瓦片索引以编码输出特征图瓦片的水平和垂直瓦片索引。

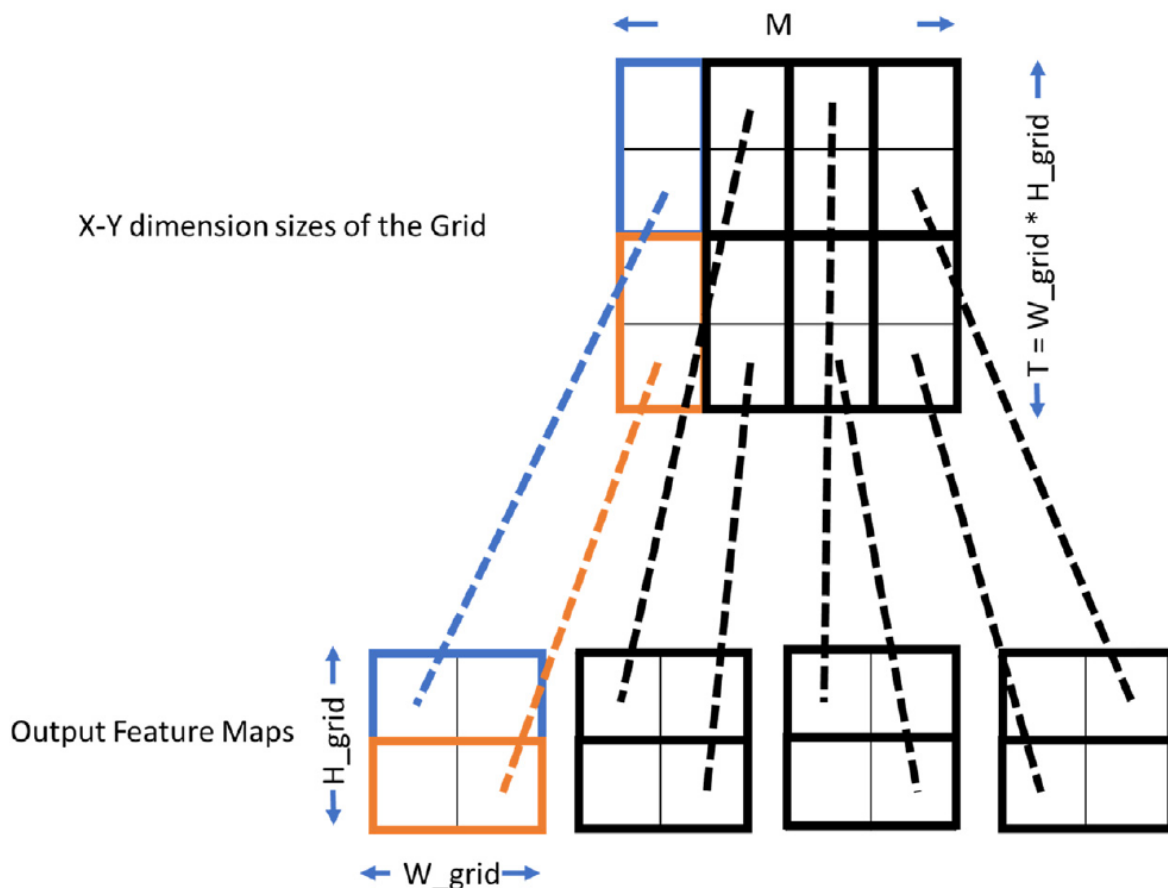


图16.14 在网格的X-Y维度上将输出特征图瓦片映射到块。

在图16.14的示例中，每个样本有四个输出特征图（ $M=4$ ），每个输出特征图由 2×2 个瓦片（ $H_grid=2$ 在第02行， $W_grid=2$ 在第03行）组成，每个瓦片有 $16 \times 16 = 256$ 像素。网格组织将每个块分配为计算这些瓦片中的一个。

我们已经将每个输出特征图分配给X维度，这反映在X维度的四个块中，每个块对应一个输出特征图。如图16.14底部所示，我们线性化每个输出特征图中的四个瓦片，并将它们分配给Y维度的块。因此，瓦片(0, 0)、(0, 1)、(1, 0)和(1, 1)分别使用行主序映射到blockIdx.y值为0、1、2和3的块。因此，Y维度的总块数是4（ $T = H_grid \times W_grid = 4$ 在第04行）。因此，我们将在第06-07行启动一个网格，其gridDim为(4, 4, N)。

```

01  __global__ void
02  ConvLayerForward_Kernel(int C, int W_grid, int K, float* X, float* W,
                           float* Y) {
03      int m = blockIdx.x;
04      int h = (blockIdx.y / W_grid)*TILE_WIDTH + threadIdx.y;
05      int w = (blockIdx.y % W_grid)*TILE_WIDTH + threadIdx.x;
06      int n = blockIdx.z;
07      float acc = 0.;
08      for (int c = 0; c < C; c++) {          // sum over all input channels
09          for (int p = 0; p < K; p++)        // loop over KxK filter
10              for (int q = 0; q < K; q++)
11                  acc += X[n, c, h + p, w + q] * W[m, c, p, q];
12      }
13      Y[n, m, h, w] = acc;
14  }

```

图16.15 卷积层前向路径的内核。

图16.15展示了一个基于上述线程组织的内核。注意，在代码中，我们使用数组访问中的多维索引以清晰明了。我们将其转换为常规C语言的练习留给读者，假设X、Y和W必须根据行主序布局（第3章，多维网格和数据）通过线性化索引进行访问。

每个线程首先生成其分配的输出特征图像素的 n （批次）、 m （特征图）、 h （垂直）和 w （水平）索引。 n （第06行）和 m （第03行）索引是直接的，由主机代码给出。对于第04行的 h 索引计算，首先将 blockIdx.y 值除以 W_grid 以恢复垂直方向上的瓦片索引，如图16.13所示。然后，这个瓦片索引通过 $TILE_WIDTH$ 扩展并加上 threadIdx.y 形成输出特征图中的实际垂直像素索引（第04行）。水平像素索引的推导类似（第05行）。

图16.15中的内核具有高度的并行性，但消耗了太多的全局内存带宽。正如第7章卷积中的卷积模式讨论，内核的执行速度将受到全局内存带宽的限制。正如我们在第7章卷积中也看到的，我们可以使用常量内存缓存和共享内存瓦片化来大幅减少全局内存流量并提高内核的执行速度。这些对卷积推理内核的优化留给读者作为练习。

16.4 将卷积层表示为GEMM

我们可以通过将卷积层表示为等效的矩阵乘法操作，然后使用CUDA线性代数库cuBLAS中的高效GEMM（通用矩阵乘法）内核，来构建一个更快速的卷积层。这种方法是由Chellapilla等人（2006年）提出的。核心思想是展开和复制输入特征图像素，以使所有需要计算一个输出特征图像素的元素被存储为这样产生的矩阵的一个连续列。这将卷积层的前向操作制定为一次大型矩阵乘法。

考虑一个小的卷积层示例，它接收 $C = 3$ 个特征图作为输入，每个特征图的大小为 $3 \times 3 \times 3$ ，并产生 $M = 2$ 个输出特征，每个输出特征的大小为 $2 \times 2 \times 2$ ，如图16.5和图16.16顶部所示，为了方便起见再次展示。它使用 $M \times C = 6$ 个滤波器组，每个滤波器组的大小为 2×2 。

这层的矩阵版本将以以下方式构建：

首先，我们将重新排列所有输入像素。由于卷积的结果在输入特征上求和，输入特征可以连接成一个大矩阵。每个输入特征图成为大矩阵中的一行部分。如图16.16所示，输入特征图0、1和2分别成为“输入特征 $X_unrolled$ ”矩阵的顶部、中间和底部部分。

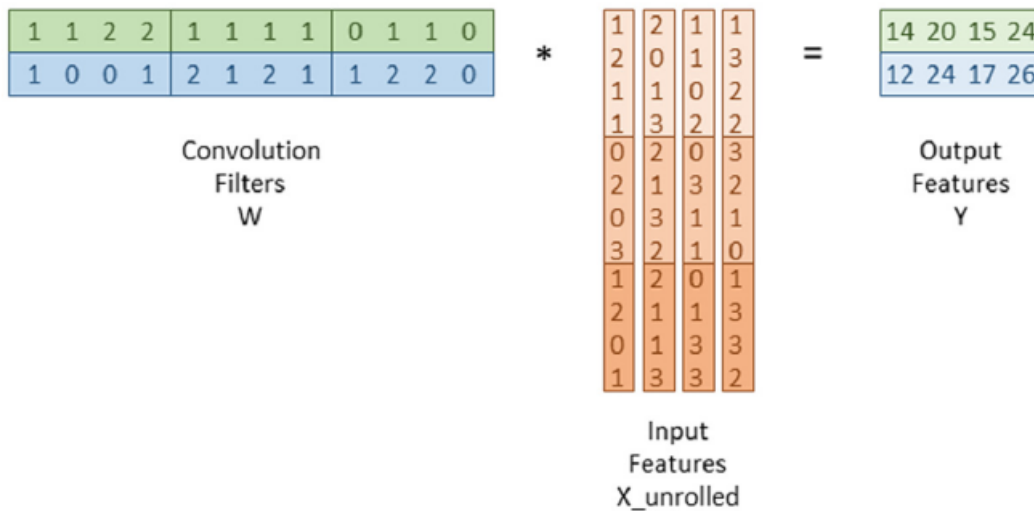
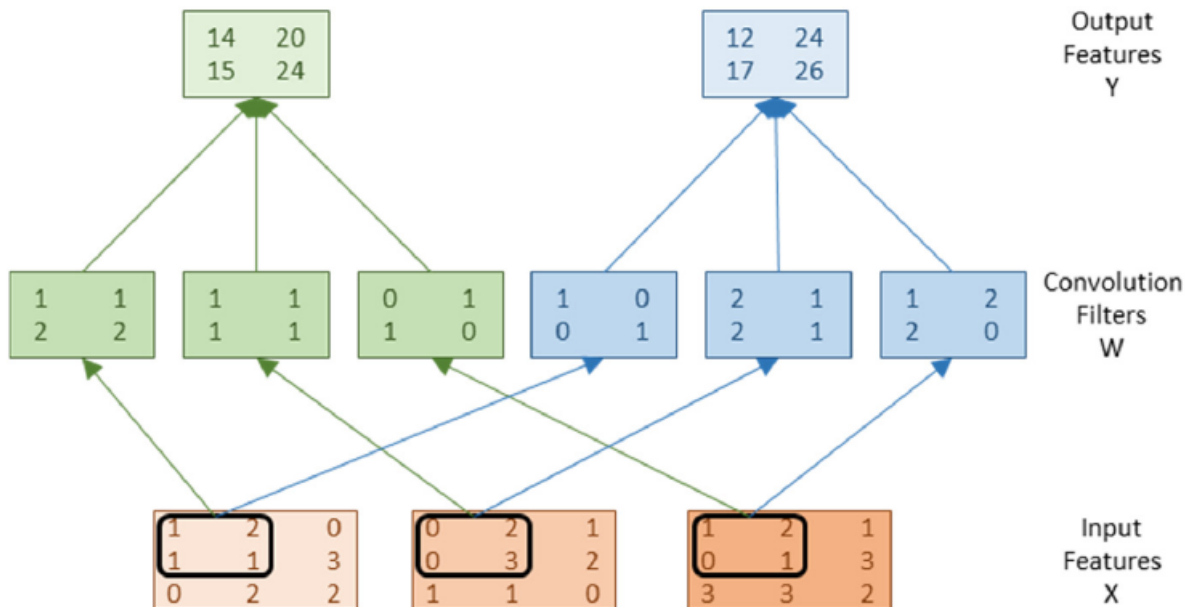


图16.16 卷积层作为GEMM的公式化。

重新排列的目的是让结果矩阵的每一列包含计算一个输出特征元素所需的所有输入值。例如，在图16.16中，所有需要计算输出特征图0在(0,0)处的值的输入特征像素在输入特征图中被圈出：

$$\begin{aligned}
 Y_{0,0,0} &= (1, 2, 1, 1) \cdot (1, 1, 2, 2) + (0, 2, 0, 3) \cdot (1, 1, 1, 1) + (1, 2, 0, 1) \cdot (0, 1, 1, 0) \\
 &= 1 + 2 + 2 + 2 + 0 + 2 + 0 + 3 + 0 + 2 + 0 + 0 \\
 &= 14
 \end{aligned}$$

其中每个内积的第一项是由线性化图16.16中圈出的x像素块形成的向量。第二项是通过线性化用于卷积的滤波器组形成的向量。在这两种情况下，线性化是按照行主序完成的。同样很明显，我们可以将三个内积重新表述为一个内积：

$$\begin{aligned}
 Y_{0,0,0} &= (1, 2, 1, 1, 0, 2, 0, 3, 1, 2, 0, 1) \cdot (1, 1, 2, 2, 1, 1, 1, 1, 0, 1, 1, 0) \\
 &= 1 + 2 + 2 + 2 + 0 + 2 + 0 + 3 + 0 + 2 + 0 + 0 \\
 &= 14
 \end{aligned}$$

如图16.16底部所示，来自滤波器组的连接向量成为滤波器矩阵的第一行，来自输入特征图的连接向量成为展开输入特征图矩阵的第一列。在矩阵乘法过程中，滤波器矩阵的一行和输入特征矩阵的一列将产生输出特征图的一个像素。

请注意， 2×12 滤波器矩阵和 12×8 输入特征图矩阵的矩阵乘法产生了一个 2×8 输出特征图矩阵。输出特征图矩阵的顶部部分是输出特征图0的线性化形式，底部是输出特征图1。它们都已经是行主序，因此可以用作下一层的单独输入特征图。至于滤波器组，滤波器矩阵的每一行只是原始滤波器组的行主序视图。因此，滤波器矩阵只是所有原始滤波器组的连接。不涉及滤波器元素的物理重新排列或移动。

我们观察到一个重要的事实，即由于卷积的性质，计算输出特征图中不同像素的输入特征图像素的补丁彼此重叠。这意味着在产生展开的输入特征矩阵时，每个输入特征图像素会被多次复制。例如，每个 $3 \times 3 \times 3$ 输入特征图的中心像素被用来计算四个输出特征图的像素，因此它将被复制四次。每个边缘上的中间像素被使用两次，因此它将被复制两次。每个输入特征图角落的四个像素只使用一次，不需要复制。因此，展开的输入特征矩阵部分中的像素总数是 $4 \times 1 + 2 \times 4 + 1 \times 4 = 16$ 。由于每个原始输入特征图只有九个像素，GEMM公式在表示输入特征图时产生了 $16/9 = 1.8$ 的扩展比率。

一般来说，展开的输入特征图矩阵的大小可以从生成每个输出特征图元素所需的输入特征图元素数量中推导出来。展开矩阵的高度，或者说行数，是贡献给每个输出特征图元素的输入特征元素的数量，这是 $C \times K \times K$ ：每个输出元素是来自每个输入特征图的 $K \times K$ 元素的卷积，共有 C 个输入特征图。在我们的示例中， K 是2，因为滤波器组是 2×2 ，有三个输入特征图。因此，展开矩阵的高度应该是 $3 \times 2 \times 2 = 12$ ，这正是图16.16中所示矩阵的高度。

展开矩阵的宽度，或者说列数，是每个输出特征图中的元素数量。如果每个输出特征图是一个 $H_{out} \times W_{out}$ 矩阵，那么展开矩阵的列数是 $H_{out} \times W_{out}$ 。在我们的示例中，每个输出特征图是一个 2×2 矩阵，在展开矩阵中产生四列。注意，输出特征图 M 的数量并不影响复制。这是因为所有输出特征图都是从相同的展开输入特征图矩阵计算出来的。

输入特征图的扩展比率是展开矩阵的大小与原始输入特征图的总大小之比。读者应该验证扩展比率如下：

$$\frac{C * K * K * H_{out} * W_{out}}{C * H_{in} * W_{in}}$$

其中 H_{in} 和 W_{in} 分别是每个输入特征图的高度和宽度。在我们的示例中，比率是 $(3 \times 2 \times 2 \times 2 \times 2) / (3 \times 3 \times 3) = 16/9$ 。一般来说，如果输入特征图和输出特征图比滤波器大得多，扩展比率将接近 $K \times K$ 。

滤波器组以完全线性化的布局表示为滤波器矩阵，其中每一行包含产生一个输出特征图所需的所有权重值。滤波器矩阵的高度是输出特征图的数量（ M ）。计算不同的输出特征图涉及共享一个展开的输入特征图矩阵。滤波器矩阵的宽度是生成每个输出特征图元素所需的权重值的数量，这是 $C \times K \times K$ 。回想一下，将权重值放入滤波器矩阵时没有复制。例如，滤波器矩阵简单地是图16.16中六个滤波器组的连接排列。

当我们将滤波器矩阵 W 与展开的输入矩阵 $X_{unrolled}$ 相乘时，输出特征图将被计算为一个高度为 M 、宽度为 $H_{out} \times W_{out}$ 的矩阵 Y 。也就是说， Y 的每一行是一个完整的输出特征图。

现在让我们讨论如何在CUDA中实现这个算法。首先让我们讨论数据布局。我们可以从输入和输出矩阵的布局开始。

1. 我们假设minibatch中的输入特征图样本将与基本CUDA内核的那些以相同的方式提供。它被组织为一个 $N \times C \times H \times W$ 数组，其中 N 是minibatch中的样本数量， C 是输入特征图的数量， H 是每个输入特征图的高度， W 是每个输入特征图的宽度。

- 正如我们在图16.16中所示，矩阵乘法将自然产生一个存储为 $M \times (H_{\text{out}} \times W_{\text{out}})$ 数组的输出 Y 。这就是原始基本CUDA内核将产生的。【译注：应该还有一个batch的维度 N 。】
- 由于滤波器矩阵不涉及权重值的复制，我们假设它将事先准备好，并组织为图16.16所示的 $M \times C \times K^2$ 数组。

展开输入特征图矩阵 X_{unroll} 的准备更为复杂。由于每次扩展将输入大小增加最多 K^2 倍，对于典型的 K 值为5或更大，扩展比率可能非常大。保持minibatch的所有样本输入特征图的内存占用可能非常大。为了减少内存占用，我们将只为 X_{unrolled} 分配一个缓冲区 $[C \times K \times K \times H_{\text{out}} \times W_{\text{out}}]$ 。我们将通过在minibatch中的样本上循环来重用这个缓冲区。在每次迭代中，我们将样本输入特征图从其原始形式转换为展开矩阵。

```

01 void unroll(int C, int H, int W, int K, float* X, float* X_unroll) {
02     int H_out = H - K + 1;
03     int W_out = W - K + 1;
04     for(int c = 0; c < C; c++) {
05         // Beginning row index of the section for channel C input feature
06         // map in the unrolled matrix
07         w_base = c * (K*K);
08         for(int p = 0; p < K; p++) {
09             for(int q = 0; q < K; q++) {
10                 for(int h = 0; h < H_out; h++) {
11                     int h_unroll = w_base + p*K + q;
12                     for(int w = 0; w < W_out; w++) {
13                         int w_unroll = h * W_out + w;
14                         X_unroll[h_unroll, w_unroll] = X(c, h + p, w + q);
15                     }
16                 }
17             }
18         }
19     }
20 }

```

图16.17 一个生成展开 X 矩阵的C语言函数。数组访问以多维索引形式呈现，以便清晰起见，需要线性化才能使代码可编译。

图16.17展示了一个按顺序产生 X_{unroll} 数组的函数，通过收集和复制输入特征图 X 的元素。该函数使用五个级别的循环。最内两层的for循环（ w 和 h ，第08-13行）为每个输出特征图元素放置一个输入特征图元素。接下来的两层（ p 和 q ，第06-14行）为每个 $K \times K$ 滤波器矩阵元素重复该过程。最外层循环重复所有输入特征图的过程。这种实现在概念上是直接的，并且可以很容易地并行化，因为循环不会在它们的迭代之间施加依赖关系。此外，最内层循环（ w ，第10-13行）的连续迭代从 X 中的一个输入特征图的一个局部瓦片中读取，并在展开矩阵 X_{unroll} 中的连续位置（ X_{unroll} 的同一行）写入。

我们现在准备设计一个CUDA内核，实现输入特征图的展开。每个CUDA线程将负责收集 $(K \times K)$ 个输入元素从一个输入特征图中为一个输出特征图元素。线程的总数将是 $(C \times H_{\text{out}} \times W_{\text{out}})$ 。我们将使用一维线程块，并从线性化的线程索引中提取多维索引。

```

01  __global__ void
02  unroll_Kernel(int C, int H, int W, int K, float* X, float* X_unroll) {
03      int t = blockIdx.x * blockDim.x + threadIdx.x;
04      int H_out = H - K + 1;
05      int W_out = W - K + 1;
06      // Width of the unrolled input feature matrix
07      int W_unroll = H_out * W_out;
08      if (t < C * W_unroll) {
09          // Channel of the input feature map being collected by the thread
10          int c = t / W_unroll;
11          // Column index of the unrolled matrix to write a strip of
12          // input elements into (also, the linearized index of the output
13          // element for which the thread is collecting input elements)
14          int w_unroll = t % W_unroll;
15          // Horizontal and vertical indices of the output element
16          int h_out = w_unroll / W_out;
17          int w_out = w_unroll % W_out;
18          // Starting row index for the unrolled matrix section for channel c
19          int w_base = c * K * K;
20          for(int p = 0; p < K; p++)
21              for(int q = 0; q < K; q++) {
22                  // Row index of the unrolled matrix for the thread to write
23                  // the input element into for the current iteration
24                  int h_unroll = w_base + p*K + q;
25                  X_unroll[h_unroll, w_unroll] = X[c, h_out + p, w_out + q];
26              }
27      }
28  }

```

图16.18 一个用于展开输入特征图的CUDA内核实现。数组访问以多维索引形式呈现，以便清晰起见，需要线性化才能使代码可编译。

图16.18展示了一个展开内核的实现。注意，每个线程将构建一个 $K \times K$ 部分的列，如图16.16中的输入特征 $X_{Unrolled}$ 数组中的阴影框所示。每个这样的部分包含所有元素，输入特征图 X 的通道 c ，需要执行相应的滤波器进行卷积操作以产生输出 Y 的一个元素。

比较图16.17和16.18的循环结构表明，图16.17中的最内两层循环已经被改为图16.18中的外层循环。这种交换允许通过多个线程并行完成收集输入元素的工作，这些输入元素是计算输出元素所需的。此外，每个线程收集所有输入特征图元素，这些元素是生成输出所需的，生成了一个合并的内存写入模式。如16.16图所示，相邻线程将在同一行中写入相邻的 X_{unroll} 元素，因为它们都垂直移动以完成它们的部分。对 X 的读取访问模式类似，可以通过检查相邻线程的 w_{out} 值来分析。我们把读取访问模式的详细分析作为练习留给读者。

一个重要的高级假设是我们将输入特征图、滤波器组权重和输出特征图保留在设备内存中。滤波器矩阵一旦准备好就存储在设备全局内存中，供所有输入特征图使用。对于minibatch中的每个样本，我们启动`unroll_Kernel`来准备展开矩阵，并启动一个矩阵乘法内核，如16.16图所示。

使用矩阵乘法实现卷积可以非常高效，因为矩阵乘法在所有硬件平台上都经过了高度优化。矩阵乘法在GPU上特别快，因为它具有每字节全局内存数据访问的高浮点运算比率。这个比率随着矩阵的增大而增加，这意味着矩阵乘法在小矩阵上效率较低。因此，当它为乘法创建大矩阵时，这种方法对卷积最有效。

正如我们前面提到的，滤波器矩阵是一个 $M \times (C \times K \times K)$ 矩阵，展开的输入特征图矩阵是一个 $(C \times K \times K) \times (H_{out} \times W_{out})$ 矩阵。注意，除了滤波器矩阵的高度之外，所有维度的大小都取决于卷积参数的乘积，而不是参数本身。虽然个别参数可能很小，但它们的乘积往往很大。例如，在卷积网络的早期层中， C 可能很小，但 H_{out} 和 W_{out} 可能很大。另一方面，在网络的末端， C 可能很大，但 H_{out} 和 W_{out} 可能很小。因此， $C \times H_{out} \times W_{out}$ 的乘积通常对所有层都很大。这意味着矩阵的大小对所有层来说往往是一致的大，因此使用这种方法的性能倾向于很高。

形成展开的输入特征图矩阵的一个缺点是它涉及将输入数据复制多达 $K \times K$ 次，这可能需要分配一个非常大的内存量。为了解决这个限制，实现如图16.16所示的X_unroll矩阵逐片展开，例如，通过形成展开的输入特征图矩阵并为minibatch的每个样本迭代地调用矩阵乘法。然而，这限制了实现中的并行性，并有时可能导致矩阵乘法太小，无法有效利用GPU。这种公式的另一个缺点是它降低了卷积的计算强度，因为除了读取X本身外，还需要写入和读取X_unroll，需要比直接方法更多的内存流量。因此，最高性能的实现有更复杂的安排，在实现展开算法时既要最大化GPU利用率，同时保持从DRAM的读取最小。我们将在下一节介绍CUDNN方法时再回到这一点。

16.5 CUDNN库

CUDNN是一个优化例程库，用于实现深度学习原语。它的设计目的是让深度学习框架更容易利用GPU。它提供了一个灵活且易于使用的C语言深度学习API，可以整齐地集成到现有的深度学习框架中（例如，Caffe、Tensorflow、Theano、Torch）。该库要求输入和输出数据驻留在GPU设备内存中，正如我们在前一节所讨论的。这个要求与cuBLAS类似。

该库是线程安全的，即其例程可以从不同的主机线程调用。前向和后向路径的卷积例程使用一个公共描述符，该描述符封装了层的属性。张量和滤波器通过不透明的描述符访问，具有使用每个维度上的任意步幅指定张量布局的灵活性。CNN中最重要的计算原语是一种特殊形式的批处理卷积。在这一部分，我们描述这种卷积的前向形式。控制这种卷积的CUDNN参数列在表16.1中。

Parameter	Meaning
N	Number of images in minibatch
C	Number of input feature maps
H	Height of input image
W	Width of input image
K	Number of output feature maps
R	Height of filter
S	Width of filter
u	Vertical stride
v	Horizontal stride
pad_h	Height of zero padding
pad_w	Width of zero padding

表16.1 CUDNN的卷积参数。请注意，CUDNN的命名约定与我们在前几节中使用的略有不同。

卷积有两个输入：

1. D是一个四维 $N \times C \times H \times W$ 张量，包含输入数据。
2. F是一个四维 $K \times C \times R \times S$ 张量，包含卷积滤波器。

输入数据数组（张量）D覆盖minibatch中的N个样本，每个样本有C个输入特征图，每个输入特征图有H行，每行有W列。滤波器覆盖K个输出特征图，C个输入特征图，每个滤波器组有R行，每个滤波器组有S列。输出也是一个四维张量O，覆盖minibatch中的N个样本，K个输出特征图，每个输出特征图有P行，每行有Q列，其中 $P = f(H; R; u; \text{pad_h})$ 且 $Q = f(W; S; v; \text{pad_w})$ ，这意味着输出特征图的高度和宽度取决于输入特征图和滤波器组的高度和宽度，以及填充和平铺选择。平铺参数u和v允许用户通过仅计算输出像素的一个子集来减少计算负载。填充参数允许用户指定将多少行或列的0条目附加到每个特征图，以改善内存对齐和/或向量化执行。

CUDNN (Chetlur等人, 2014) 支持多种算法实现卷积层：基于矩阵乘法的GEMM (Tan等人, 2011) 和 Winograd (Lavin & Scott, 2016)、基于FFT的 (Vasilache等人, 2014) 等。基于GEMM的算法使用矩阵乘法实现卷积，类似于第16.4节中介绍的方法。正如我们在16.4节末讨论的，将展开的输入特征矩阵在全局内存中实现可能在全局内存空间和带宽消耗方面代价高昂。CUDNN通过惰性生成并仅将展开的输入特征图矩阵 `X_unroll` 加载到片上内存中，而不是在调用矩阵乘法例程之前在片外内存中收集它，从而避免了这个问题。NVIDIA提供了一个基于矩阵乘法的例程，实现了GPU上最大理论浮点吞吐量的高利用率。

这种常规的算法类似于Tan等人 (2011) 所描述的算法。输入矩阵A和B的固定大小子矩阵依次读入片上内存，然后用于计算输出矩阵C的子矩阵。所有由卷积强加的索引复杂性都在此常规中通过瓦片的管理来处理。我们在从片外内存将A和B的下一个瓦片获取到片上缓存和其他内存的同时，计算A和B的瓦片。这种技术隐藏了与数据传输相关的内存延迟，允许矩阵乘法计算仅受执行算术计算所需时间的限制。

由于矩阵乘法常规所需的瓦片化与卷积的任何参数无关，`X_unroll`的瓦片边界与卷积问题的映射是非平凡的。因此，CUDNN方法需要计算这个映射并使用它将A和B的正确元素加载到片上内存中。随着计算的进行，这种情况会动态发生，这允许CUDNN卷积实现利用优化的矩阵乘法基础设施。与矩阵乘法相比，它需要额外的索引算术，但它充分利用了矩阵乘法的计算引擎来完成工作。计算完成后，CUDNN执行所需的张量转置，将结果存储在用户所需的数据布局中。

16.6 总结

本章从对机器学习的简要介绍开始。然后更深入地探讨了分类任务，并介绍了感知器，这是一种线性分类器，对于理解现代CNN至关重要。我们讨论了前向推理和后向传播训练过程在单层和多层感知器网络中的实现方式。特别地，我们讨论了在训练过程中如何通过链式法则更新多层感知器网络中的模型参数，以及可微分激活函数的必要性。基于对感知器的概念和数学理解，我们提出了一个基本的卷积神经网络及其主要类型层的实现。这些层可以被视为感知器的特殊情况或简单适应。然后，我们在第七章“卷积”中对卷积模式进行了构建，提出了CNN中最计算密集的层——卷积层的CUDA内核实现。

接着，我们介绍了将卷积层表述为矩阵乘法的技术，通过将输入特征图展开成矩阵。这种转换允许卷积层从高度优化的GEMM库中受益，这些库是为GPU设计的。我们还介绍了输入矩阵的展开过程的C语言和CUDA实现，并讨论了展开方法的优缺点。

本章最后概述了CUDNN库，这是大多数深度学习框架使用的库。这些框架的用户可以从高度优化的层实现中受益，而无需自己编写CUDA内核。